

Binarna klasifikacija resnosti COVID-19 okužbe s pomočjo CT slik prsnega koša - drugi del

11. december 2025

1 Uvod

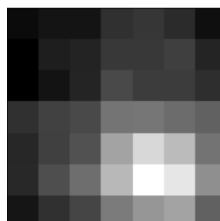
Na konvolucijske nevronske mreže se velikokrat gleda kot na črne škatle, ki sprejmejo na vходу sliko in nam na koncu dajo nek odgovor. Temu ni povsem tako. Ko je naš model naučen, so uteži posameznih plasti zaklenjene in se ne spreminjajo več. Vemo tudi, kakšne so vrednosti teh uteži in kakšna je povezava med posameznimi plastmi. Torej bi načeloma za vsako sliko morali vedeti, kaj se z njo dogaja, ko potuje skozi mrežo. Je pa seveda res, da z večanjem števila filtrov, konvolucij in plasti interpretabilnost našega modela pada.

Možnosti za interpretacijo konvolucijskih nevronskih mrež je več. V tej nalogi boste spoznali tri osnovne pristope.

1.1 Vizualizacija filtrov

Predvsem filtri začetnih plasti konvolucijske nevronske mreže nam lahko povejo veliko o tem, kakšne slikovne značilke poskuša naš model izluščiti iz vhodne slike. Ponavadi so v začetnih plasteh ti filtri preprosti, iščejo oblike, linije ipd. Začetni filtri so tako najbolj razločljivi, saj je rezultat konvolucije odvisen od vhodne slike, ki jo poznamo in razumemo. Tem globlje kot gremo skozi konvolucijsko nevronske mrežo tem bolj so rezultati konvolucij abstraktni. V Pytorchu filtre preprosto vizualiziramo s klicem atributa `weight.data` na izbrani konvolucijski plasti (skripta `visualize_filters.py`):

```
1
2 # npr. za modificiran ResNet18, se prva plast skriva v backbone.conv1
3
4 weights = model.backbone.conv1.weight.data.cpu().numpy()
5 # weights.shape = (64,1,7,7) -> 64 filtrov, z 1 kanalom, velikosti 7x7
6
7 # Vizualizacija 21. filtra iz prve plasti (backbone.conv1)
8 plt.imshow(weights[20,0,:,:], cmap='gray')
```

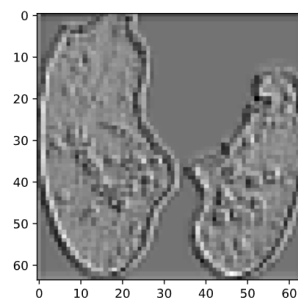


Slika 1: Prikaz 21. filtra iz prve konvolucijske plasti.

1.2 Vizualizacija konvolucij

Če želimo razumeti, kaj se z našo sliko dogaja med propagacijo skozi konvolucijsko nevronske mrežo, moramo naš model najprej postaviti v fazo evalvacije `model.eval()`. Ko skozi naš model propagiramo sliko, si preprosto sproti shranjujemo rezultate posameznih konvolucij. En izmed načinov, ki temelji na t.i. *hookih*, je prikazan spodaj (skripta `visualize_convolution.py`).

```
1 import ... # glej skripto
2 # Razred za lažjo shranjevanje outputov, ki temelji na principu kavljev (hookov)
3 class SaveOutput:
4     def __init__(self):
5         self.outputs = []
6     def __call__(self, module, module_in, module_out):
7         self.outputs.append(module_out)
8     def clear(self):
9         self.outputs = []
10
11 # Nalozimo model
12 model = ModelCT()
13 model.to(device)
14 model.load_state_dict(torch.load(os.path.join(model_folder, "trained_model_weights.
15                                    .pth")))
16 model.eval()
17
18 # Inicializiramo razred v SO in registriramo kavlje v našem modelu
19 SO = SaveOutput()
20 for layer in Model.modules(): # hooke postavimo na vse plasti, ki so tipa Conv2d
21     if isinstance(layer, torch.nn.modules.conv.Conv2d):
22         handle = layer.register_forward_hook(SO)
23
24 # Naredimo test_generator z batch_size=1
25 test_datareader = DataReader(main_path_to_data, test_info)
26 test_generator = DataLoader(test_datareader, batch_size=1, shuffle=False, pin_memory=
27                               True, num_workers=2)
28
29 # Vzamemo en testni primer in ga poženemo skozi mrežo
30 item_test = next(iter(test_generator))
31 input_image = item_test[0].to(device)
32 _ = model(input_image)
33
34 # Izberemo katero konvolucijo bi radi pogledali (color_channel bo vedno 0)
35 color_channel = 0 # indeks barvnega kanala (pri nas le 1 kanal)
36 idx_layer = 5 # indeks konvolucijske plasti (Conv2d) - (pri nas 21)
37 idx_convolution = 17 # indeks konvolucije na dani plasti (max odvisen od plasti)
38
39 # Vizualiziramo
40 X = SO.outputs[idx_layer][color_channel][idx_convolution][:,:].cpu().detach().numpy()
41 plt.imshow(np.rot90(X), cmap='gray')
```



Slika 2: Rezultat konvolucije z 18. filtrom na 6. konvolucijski plasti.

1.3 Mapa izpostavljenosti - Saliency maps

Mape izpostavljenosti oz. *saliency maps* nam povedo, kateri piksli vhodne slike prinašajo največ h končni odločitvi konvolucijske nevronske mreže.

1.3.1 Teoretično ozadje

Za dano sliko I_0 in naučeno konvolucijsko nevronske mrežo S bi radi piksle slike I_0 rangirali glede na njihov vpliv na končno verjetnost, da naša slika pripada razredu z **hudo** okužbo (oznaka 1), kar označimo z $S(I_0)$.

Preprosti primer, bi bil npr:

$$S(I) = w^T I + b, \quad (1)$$

kjer je slika I predstavljena kot enodimenzionalni vektor in w ter b zaporedoma uteži in bias našega modela. V tem primeru lahko lepo vidimo, da velikost uteži w predstavlja, kakšen je doprinos posameznih pikslov h končni odločitvi. V primeru globokih konvolucijskih nevronske mreže, je vrednost $S(I)$ močno ne-linearna funkcija, zato logike prejšnjega primera ne moremo direktno aplicirati. Ampak, za dano sliko I_0 , lahko $S(I)$ aproksimiramo z linearno funkcijo v okolici I_0 tako, da izračunamo prvi red Taylor-jevega razvoja:

$$S(I) \approx w^T I + b, \quad (2)$$

kjer w odvod S -ja glede na sliko I v točki (sliki) I_0

$$w = \left. \frac{\partial S}{\partial I} \right|_{I_0}. \quad (3)$$

Tukaj nam vrednost odvoda pove, kakšno spremembo na izhodu povzroči sprememba vrednosti piksla. Za dano sliko I_0 z m vrsticami in n stolpci lahko mapo izpostavljenosti $M \in \mathbb{R}^{m \times n}$ izračunamo tako, da najprej z vzvratno propagacijo poiščemo odvod w (3). Mapo izpostavljenosti M dobimo tako, da prerazporedimo elemente vektorja w . V primeru črno-bele slike (en vhodni kanal) je število elementov v w enako številu pikslov slike I_0 , tako da mapo M izračunamo kot

$$M_{ij} = |w_{h(i,j)}|, \quad (4)$$

pri čemer $h(i,j)$ indeks elementa w , ki pripada vrednosti piksla slike v i -ti vrstici in j -tem stolpcu.

1.3.2 Primer uporabe

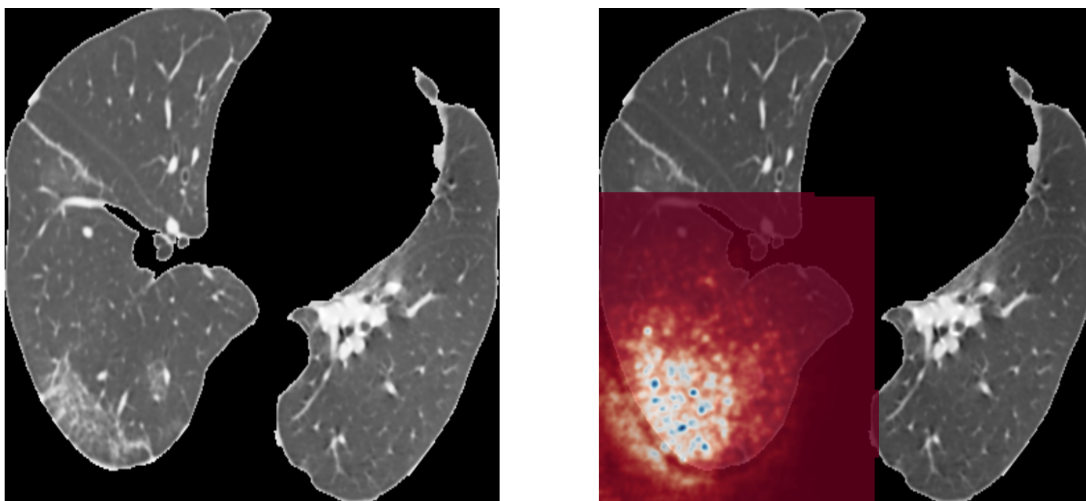
Captum je knjižnica za interpretacijo nevronske mreže, zgrajenih v okolju Pytorch. Omogoča implementacijo številnih tehnik, ki slonijo na različnih matematičnih ozadjih in so iz vidika računske/spominske kompleksnosti med seboj zelo razlikujejo. Kot že rečeno, si bomo ogledali t.i. mape izpostavljenosti oz. *saliency maps*. Primer uporabe je predstavljen v skripti `visualize_saliency.py`. Vizualizirana je originalna slika `original` z očitnim območjem mlečnega stekla (levi spodnji kot) in pripadajoča mapa izpostavljenosti `attribution` (Slika 3).

```
1 import ... # glej skripto
2 from captum.attr import Saliency
3 from scipy.ndimage import gaussian_filter
4
5 model = ... # glej skripto
6
7 # Iz knjižnice captum naložimo Saliency
8 saliency = Saliency(model)
9
10 # Poiščemo primer z dobro klasifikacijo hude okužbe (y==1, y_hat > 0.95)
```

```

11 for item_test in test_generator:
12     x, y = item_test
13     x = x.to(device)
14     y = y.to(device)
15
16     # Forward pass
17     y_hat = model.forward(x)
18     y_hat = torch.sigmoid(y_hat)
19
20     if int(y) == 1 and float(y_hat) > 0.95:
21         attribution = saliency.attribute(x)
22         attribution = np.rot90(attribution.detach().cpu().numpy().squeeze())
23         original = np.rot90(x.cpu().numpy().squeeze())
24
25         # Po zelji zgladimo saliency z gaussom
26         attribution = gaussian_filter(attribution, sigma=2)

```



Slika 3: Primer pravilne klasifikacije CT rezine v razred hude okužbe. Konvolucijska nevronska mreža je odločitev sprejela večinsko na podlagi mlečnega stekla. Levo: originalna slika z mlečnim steklom v spodnjem levem kotu. Desno: z gaussovim filtrom zglajena mapa izpostavljenosti.

2 Navodilo drugega dela naloge

V prejšnjem delu naloge ste na podlagi validacijske množice izbrali 3 različne končne modele. Za vašo konvolucijsko nevronska mrežo in prednaučen ResNet18 storite sledeče:

1. Vizualizirajte in **na kratko** interpretirajte filtre vhodne konvolucijske plasti.
2. Prikažite, kaj se z vhodno sliko dogaja na različnih globinah mreže (1 primer).
3. Iz testne množice poiščite nekaj primerov napačne klasifikacije ter vizualizirajte relativni doprinos pikselov h končni odločitvi (*saliency maps*).